GO: MODULI, TOOL

PACKAGE



PACKAGE

- Un programma Go è una composizione di **pacchetti**.
- Il pacchetto principale è `main`.
- La funzione iniziale (punto d'ingresso) è `main()` all'interno del pacchetto `main`.
- I pacchetti vengono importati utilizzando il loro **percorso**, ad esempio `encoding/json` o `fmt`.
- Ogni pacchetto deve risiedere in una **directory dedicata**.
- Un **modulo** è un gruppo di pacchetti.



MODULE

- Collezione di **uno o più pacchetti**.
- Unità di **Gestione delle Dipendenze** e Controllo di Versione.
- Definito dal file `go.mod`.
- Importazione: Usa il percorso completo (`nome-modulo/pacchetto`).
- Comando: `go mod init nome-modulo` crea la radice del progetto.



UTILIZZARE I PACCHETTI

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    // Stampa un numero casuale tra 0 e 9
    fmt.Println("Il mio numero preferito è", rand.Intn(10))
}
```

LIBRERIA STANDARD VS. ESTERNA

- Go fornisce una **libreria standard** con pacchetti comuni: `https://pkg.go.dev/std`.
- Puoi creare pacchetti anche **all'interno del tuo progetto**.
- Puoi usare pacchetti esterni o crearne di nuovi da condividere.

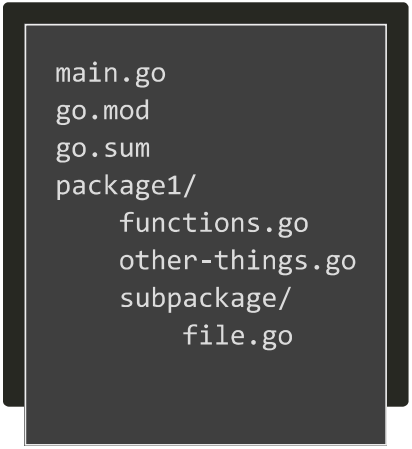


CREARE PACCHETTI ALL'INTERNO DEL TUO PROGETTO

STRUTTURA DEL PROGETTO

1. All'interno della directory del tuo progetto, `go mod init nome-modulo` crea un modulo (`go.mod` e `go.sum`).
2. A partire da quel momento, puoi creare **sotto-moduli** (sotto-directory) per i pacchetti.
3. Li importi usando il percorso completo: `nome-modulo/sotto-dir`.




main.go
go.mod
go.sum
package1/
functions.go
other-things.go
subpackage/
file.go

ESEMPIO: PACCHETTO INTERNO

Contenuto di `package1/functions.go`:

```go
package package1

// Nota: Dobbiamo usare la lettera MAIUSCOLA come prima
// lettera per rendere questa funzione pubblica.
// Altrimenti sarà disponibile solo all'interno di package1.
func Dummy() {}

// Questa funzione è privata (non esportata)
func internalFunction() {}
```



Contenuto di `main.go`:

```
package main

import "nome-modulo/package1" // Importa il pacchetto interno

func main() {
    // Chiama la funzione pubblica Dummy
    package1.Dummy()
    // package1.internalFunction() // Errore: non è esportata
}
```



MODULI/PACCHETTI ESTERNI



OTTENERE DIPENDENZE ESTERNE

Per utilizzare moduli esterni, devi prima "ottenerli" (scaricarli).

Comando:

```
$ go get gopkg.in/yaml.v2
```

Questo comando scarica il pacchetto `gopkg.in/yaml.v2` e lo aggiunge a `go.mod`

e si crea il file go.sum

- [ ]

Esempio di importazione e utilizzo:

```go
package main

import "gopkg.in/yaml.v2" // Pacchetto esterno

func main() {
    data := map[string]string{
        "name": "Go",
        "type": "Language",
    }

    // Marshalling (conversione in YAML)
    yamlData, _ := yaml.Marshal(data)
    // fmt.Println(string(yamlData))
}
```

GO TOOLCHAIN



CREARE E DEFINIRE IL MODULO

Il comando `go mod` è fondamentale per inizializzare e gestire il progetto.

Comando	Descrizione
`go mod init <path/nome>`	**Inizializza un nuovo modulo Go.** Crea il file `go.mod` nella directory corrente, definendo il percorso radice del modulo (es. `github.com/utente/repo`).
`go mod tidy`	**Pulisce e Aggiorna.** Aggiunge eventuali dipendenze mancanti (trovate nel codice) e rimuove quelle inutilizzate dal file `go.mod` e `go.sum`.
`go mod download`	Scarica tutti i moduli richiesti in `go.mod` nella cache locale.

STRUTTURA STANDARD (LAYOUT)

I progetti Go seguono una convenzione semplice:

- **File Sorgente:** I file Go finiscono con `.go`.
- **Package `main`:** Contiene la funzione `main()` ed è il punto d'ingresso per un programma eseguibile.
- **Package (Nome Personalizzato):** Contiene librerie e logica di business riutilizzabile.



Esempio:

1. Crea la cartella: `mkdir myproject && cd myproject`
2. Inizializza il modulo: `go mod init example.com/myproject`
3. Crea il file principale: `touch main.go`



RUN ED ESECUZIONE RAPIDA

Per eseguire rapidamente il codice senza produrre un eseguibile finale.

Comando	Descrizione
`go run main.go`	**Compila ed esegue** il codice sorgente specificato. Utile per testare rapidamente durante lo sviluppo.
`go run .`	Compila ed esegue il package `main` nella directory corrente.



COMPILAZIONE E BUILD

Per creare un eseguibile binario distribuibile.

Comando	Descrizione
`go build .`	**Compila** il package corrente (o `main` se eseguibile) e genera un file binario nella directory corrente, chiamato come il modulo o la directory.
`go build -o app_name .`	Compila e specifica il nome del file di output (es. `app_name`).
`go install .`	Compila e installa l'eseguibile nella cartella binari predefinita di Go (`$GOPATH/bin` o `$GOBIN`).

TEST

Il framework di testing è integrato nella toolchain.

Comando	Descrizione
`go test`	**Esegue i test** (file con suffisso `_test.go`) nella directory corrente.
`go test ./...`	Esegue i test in tutte le sottodirectory.
`go test -v`	Esegue i test in modalità verbosa (mostra i dettagli di ogni test).



FORMATTAZIONE DEL CODICE

Go impone uno stile di formattazione standardizzato.

Comando	Descrizione
`go fmt .`	**Riformatta automaticamente** tutti i file Go nella directory corrente in modo standardizzato, utilizzando lo stile canonico di Go.



ANALISI E LINTING (CONTROLLO QUALITÀ)

Strumenti integrati per l'analisi statica del codice.

Comando	Descrizione
`go vet .`	**Analizza il codice sorgente** per identificare potenziali errori (bug) o costrutti sospetti, come errori comuni di formattazione di stringhe o assegnazioni inutili.
`go doc <package/func>`	Mostra la documentazione per un pacchetto o una funzione specifici. (es. `go doc fmt.Println`).



ALTRI STRUMENTI UTILI

Comandi per la gestione e l'ispezione dell'ambiente.

Comando	Descrizione
`go get <path/package>`	Aggiunge un nuovo pacchetto esterno al file `go.mod` e lo scarica.
`go version`	Mostra la versione di Go installata.
`go env`	Stampa le variabili d'ambiente di Go (es. `$GOPATH`, `$GOBIN`).



AGGIUNTA E AGGIORNAMENTO

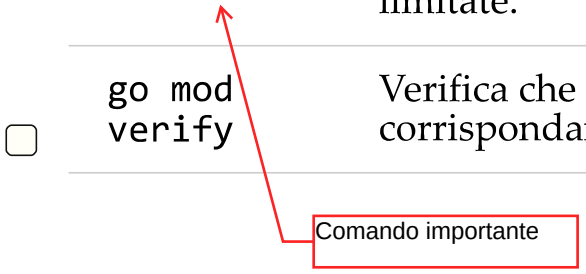
I comandi gestiscono le dipendenze esterne richieste dal codice.

Comando	Descrizione
`go get <path/package>`	Aggiunge una nuova dipendenza al progetto (o aggiorna una esistente) e la registra in `go.mod`.
`go get <path/package>@v1.2.3`	Scarica e utilizza una specifica versione o un tag.
`go get -u ./...`	**Aggiorna tutte le dipendenze** del modulo alla versione patch o minor più recente.
`go get -u=patch ./...`	Aggiorna solo alle versioni patch più recenti (più sicuro).

ISPEZIONE E BLOCCO (GO.SUM)

Il file `go.sum` contiene gli hash crittografici per le dipendenze, garantendo l'integrità.

Comando	Descrizione
`go list -m all`	**Elenca tutte le dipendenze** del modulo, incluse quelle transitive (di cui dipendono le tue dipendenze).
`go mod vendor`	Crea una cartella `vendor/` contenente le copie locali di tutte le dipendenze, per build isolate o per reti limitate.
`go mod verify`	Verifica che i moduli scaricati nella cache Go corrispondano agli hash in `go.sum`.



PULIZIA

Per mantenere pulito l'ambiente del modulo.

Comando	Descrizione
`go clean -modcache`	**Cancella la cache dei moduli** (`$GOPATH/pkg/mod`), forzando il download di tutte le dipendenze alla prossima build/run.



